

Práctica C. TDD: JUnit y Jacoco

Objetivos

- Desarrollar código para implementar HU utilizando pruebas unitarias y de cobertura.
- Conocer la forma en que se diseñan, ejecutan y corrigen las pruebas unitarias.

Recursos

1. Vídeo sobre TDD, JUnit y Jacoco para esta práctica: <https://youtu.be/PtX-Ba7puec>
2. Repositorio de recursos para la práctica::
<https://docencia-vrivas.github.io/dagil-resrepo/?temas=TDD+y+Jacoco>

Tareas a realizar

Para esta práctica, aunque vamos a seguir trabajando con las mismas HU de las anteriores, es recomendable comenzar un proyecto totalmente nuevo.

Primera fase: preparar el entorno de trabajo teniendo en cuenta que usaremos TDD

1. Crea un nuevo proyecto en Java usando el arquetipo de *maven* **archetype-quickstart-jdk8**
2. Comprueba que puedes compilarlo y ejecutarlo.
 - a. Ten en cuenta que es posible que tengas instalar el plugin *Extension Pack for Java*, para VS Code, así como la aplicación **maven** en el ordenador.
3. Crea el fichero **.gitignore** tal como se muestra en el vídeo de la práctica.
4. Modifica el fichero **pom.xml** tal como se muestra en el vídeo de la práctica.
5. Comprueba que puedes ejecutar las fases *clean* y *test* desde *maven*.
6. Inicializa la rama **main** del repositorio y haz el *commit* inicial.
7. Crea la rama **doc** y añade en ella el fichero **README.md** inicial No olvides volver a actualizar la rama **main** con el contenido de la rama **doc**

Con los pasos anteriores, ya tendrías configurado tu entorno de desarrollo.

Segunda fase: implementar y documentar las HU

A continuación, vamos a llevar a cabo el desarrollo de la aplicación, pero implementando solamente un incremento con las siguientes funcionalidades:

HU8 Dar de alta un nuevo registro horario (con datos válidos).

HU11. Mostrar en pantalla todos los registros horarios.

HU12. Dados dos registros horarios de una misma persona en una misma empresa, siendo el primero de ellos de entrada y el segundo de salida, mostrar la diferencia que hay entre ambos en formato de horas, minutos y segundos. Para ello, escribir un método en la clase RegistroHorario que tendrá la siguiente especificación:

```
/**  
 * Calcula la diferencia en horas, minutos y segundos entre este  
 registro horario que recibe el mensaje y el registro que se le pasa  
 como parámetro,  
 * @param registroSalida El registro de salida con respecto al cual se  
 va a calcular la diferencia.
```

```
* @throws IllegalArgumentException Si el registro que recibe este  
mensaje NO es de entrada.  
* @throws IllegalArgumentException Si registroSalida NO es de salida.  
* @throws IllegalArgumentException Si el id de la persona no coincide  
en ambos registros.  
* @throws IllegalArgumentException Si el id de la empresa no coincide  
en ambos registros.  
* @throws IllegalArgumentException Si el tiempo reflejado en el  
registro de salida es anterior al registro de entrada.  
* @return Un ArrayList<Integer> donde se almacena, en este orden:  
horas, minutos y segundos de diferencia entre ambos registros.  
*/
```

Te recuerdo que los datos que formaban un registro horario son:

- *id* de la persona
- *id* de la empresa (guardamos este dato aquí porque una persona puede haber trabajado en varias empresas a lo largo del tiempo; no de forma simultánea, pero sí unas temporadas en una empresa y otras temporadas en otras).
- *tipo* de evento (si es que la persona entra o si es que la persona sale)
- *fecha* y *hora* en que se produce el evento (día, mes, año, hora, minutos y segundos).

Para llevarlo a cabo, debes crear la clase *RegistroHorario* solo con las propiedades y métodos que necesites para implementar las distintas HU que lo utilizan. **Todos los métodos que implementes de esta clase deben estar probados con TDD y debes tener una cobertura del 100% con Jacoco.**

IMPORTANTE: NO IMPLEMENTES TODA LA CLASE COMPLETA DESDE EL PRINCIPIO. Para cada HU debes crear una rama e implementar SOLO las propiedades y métodos de la clase *RegistroUsuario* que necesites para esa HU y que aún no estén implementados. Por supuesto, también debes implementar sus pruebas.

Puedes hacer la documentación cuando acabes cada HU o cuando acabes el incremento completo.

Por último, no olvides crear la rama release, así como una etiqueta para la versión de la aplicación que acabas de crear (que, si has partido de un proyecto nuevo, debe ser la versión 1.0).